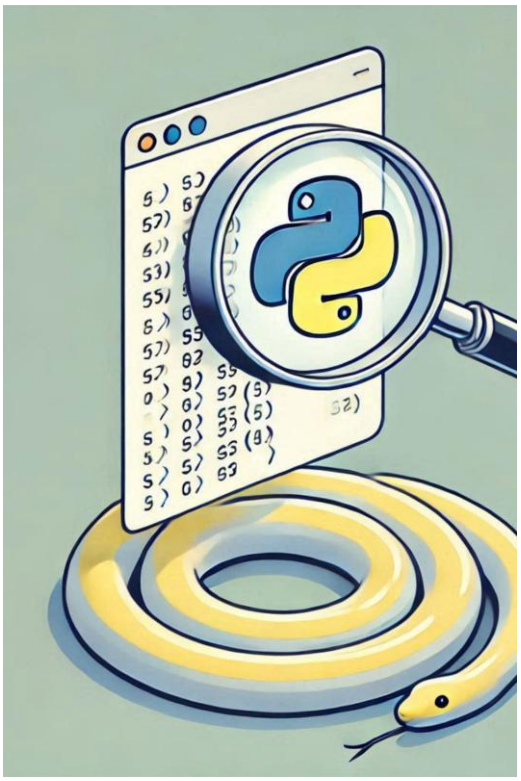
Wstęp do programowania- wykład 3

1. Obiekt i typ

👉 Obiekt – przechowuje określoną wartość.

👉 Typ danych – określa rodzaj przechowywanych wartości i działania, które można na nich wykonywać.

2. Podstawowe typy danych


pl.pinterest.com/pin/755338168786700152/

- **str – typ tekstowy**

👉 Przechowuje tekst (łańcuch znaków), np. "Ala".

👉 + – łączy teksty, * – powiela tekst.

👉 Teksty można porównywać operatorami ==, !=, <, <=, >, >=.

```
imie = "Ala"
print("Hej " + imie + "!")
print("XO" * 3)
```

- **int – liczby całkowite**

👉 Przechowuje liczby całkowite, np. 5, 0, -23.

👉 Działania: +, -, *, /, ** (potęgowanie).

- **float – liczby zmiennoprzecinkowe**

👉 Przechowuje liczby z częścią ułamkową, np. 2.5, 3.14.

👉 W Pythonie używamy kropki dziesiętnej.

👉 Liczby int i float można porównywać operatorami ==, !=, <, <=, >, >=.

👉 / zawsze daje wynik typu float.

👉 float ma ograniczoną dokładność, dlatego wyniki niektórych działań mogą być niedokładne.

3. Sprawdzanie i konwersja typów

👉 type() – sprawdza typ wartości.

👉 str() – konwersja na tekst.

👉 int() – konwersja na liczbę całkowitą.

👉 float() – konwersja na liczbę zmiennoprzecinkową.

Python może też automatycznie konwertować typy, np. przy działaniu 5 + 2.5 wynik to 7.5.

!Ważne: input() pobiera dane jako tekst (str). Aby wykonywać obliczenia, często trzeba użyć int(input()) lub float(input()) .

F-stringi – formatowane łańcuchy znaków

👉 **F-stringi** umożliwiają wstawianie wartości zmiennych i wyrażeń bezpośrednio do tekstu. Tworzymy je, dodając literę f przed cudzysłowem, a wyrażenia zapisujemy w klamrach {}.

```
imie = "Ala"

print(f"Cześć, {imie}!")

print(f"2 + 2 = {2 + 2}")
```

Formatowanie tekstu i liczb

• W klamrach {} po dwukropku : możemy określić sposób wyświetlania wartości:

👉 < – wyrównanie do lewej,

👉 > – wyrównanie do prawej,

👉 ^ – wyśrodkowanie,

👉 liczba – szerokość pola w znakach,

👉 znak wypełnienia – znak uzupełniający wolne miejsce.

Przykład: f"{'Ala':>10}" – tekst wyrównany do prawej w polu o szerokości 10 znaków.

• Przy liczbach rzeczywistych możemy określić liczbę miejsc po przecinku, używając ., liczby miejsc i litery f.

```
liczba = 5.0

print(f"{liczba:10.2f}")
```

👉 Liczba zostanie wyświetlona w polu o szerokości 10 znaków, z dwoma miejscami po przecinku.

F-stringi z =

👉 Znak = wewnątrz klamer pozwala wyświetlić wyrażenie, znak równości i jego wartość. Przydaje się to podczas szukania błędów w programie.

```
print(f"{2 + 2=}")
```

Wynik: 2 + 2=4

!Ważne: litera f przed cudzysłowem odróżnia f-string od zwykłego łańcucha znaków.

Łańcuchy wielolinijkowe

👉 to teksty zawierające więcej niż jedną linię. Tworzymy je za pomocą potrójnych cudzysłowów pojedynczych (''') lub podwójnych (""""). Mogą być również f-stringami.

```
tekst = """Pierwsza linia

Druga linia"""
```

Instrukcje warunkowe if, else, elif

👉 if – wykonuje kod, gdy warunek jest prawdziwy (True),

👉 else – wykonuje kod, gdy warunek jest fałszywy (False).

```
if wiek >= 18:
    print("Pełnoletni")
else:
    print("Niepełnoletni")
```

👉 elif to skrót od else if. Warunki są sprawdzane po kolei, a wykonywany jest kod pierwszego spełnionego warunku. else jest opcjonalne i wykonuje się, gdy żaden wcześniejszy warunek nie jest prawdziwy.

```
if ocena >= 5:
    print("Bardzo dobrze")
elif ocena >= 3:
    print("Zaliczone")
else:
    print("Niezaliczone")
```

!Ważne: wcięcia określają, które instrukcje należą do danego bloku. Bloki if, elif i else muszą mieć odpowiednie, spójne wcięcia.

4. Własne funkcje

👉 Funkcja to wydzielony fragment programu wykonujący określone zadanie. Można ją wielokrotnie wywoływać, przekazując różne dane wejściowe, czyli argumenty.

👉 Funkcję definiujemy za pomocą słowa kluczowego def, nazwy, nawiasów i dwukropka. Kod funkcji zapisujemy z wcięciem.

```
def powitanie(imie):
    """Wyświetla powitanie."""
    print(f"Cześć, {imie}!")
```

```
powitanie("Ala")
```

👉 Docstring to opis funkcji zapisany jako łańcuch znaków, zwykle na początku jej kodu. Tradycyjnie zapisuje się go w potrójnych cudzysłowach. Pomaga zrozumieć, co robi funkcja.

Funkcje – argumenty i return

👉 Argumenty – dane przekazywane do funkcji w nawiasach.

👉 Docstring – opis działania funkcji, który można odczytać m.in. za pomocą help().

👉 Adnotacje typów – zapisujemy po argumencie dwukropek i typ, a po nawiasie zamykającym argumenty strzałkę -> i typ wyniku.

👉 return – kończy działanie funkcji i zwraca wynik do miejsca wywołania. Samo nie wyświetla nic na ekranie.

👉 print() – wyświetla dane na ekranie.



pl.pinterest.com/pin/818810776032815122/

```
def dodaj(a: int, b: int) -> int:
    return a + b

wynik = dodaj(2, 3)

print(wynik)
```

```
5
```

Instrukcja assert

👉 Służy do testowania programu i sprawdzania, czy warunek jest prawdziwy.

👉 Jeśli warunek jest prawdziwy, nic się nie dzieje.

👉 Jeśli jest fałszywy, pojawia się wyjątek AssertionError.

👉 Używamy jej do wykrywania błędów programu, a nie do sprawdzania poprawności danych od użytkownika.

```
assert 2 + 2 == 4
```

Listy

👉 Lista to uporządkowana kolekcja elementów, które można dodawać, usuwać i zmieniać.

- Zapisujemy ją w nawiasach kwadratowych [].

- Indeksowanie zaczyna się od 0.

- Ostatni element ma indeks len(lista) - 1.

- len(lista) – zwraca liczbę elementów.

- lista.append(element) – dodaje element na końcu listy; nie zwraca nowej listy.

```
owoce = ["jabłko", "gruszka"]

owoce.append("banan")
```

```
print(owoce[0])
```

```
jabłko
```

👉 Pętla for pozwala przechodzić kolejno przez elementy listy (iterować po liście).

```
for owoc in owoce:
    print(owoc)
```

Metody łańcuchów znaków

👉 split() – dzieli tekst na listę podłańcuchów; domyślnie dzieli według białych znaków.

👉 join() – łączy łańcuchy z listy, używając wskazanego separatora.

```
tekst = "Ala ma kota"
print(tekst.split())
```

```
['Ala', 'ma', 'kota']
```



pl.pinterest.com/pin/954622452288458114/

```
slowa = ["Ala", "ma", "kota"]
print(" ".join(slowa))
```

```
Ala ma kota
```

Słowniki

👉 Słownik (dict) przechowuje pary klucz-wartość. Klucze służą do odnajdywania przypisanych im wartości.

👉 Słownik zapisujemy w nawiasach klamrowych {}, a klucz od wartości oddzielamy dwukropkiem :.

👉 Pary klucz-wartość oddzielamy przecinkami.

👉 slownik[klucz] – odczytuje wartość przypisaną do klucza.

👉 slownik[klucz] = wartosc – dodaje parę lub zmienia istniejącą wartość.

👉 in – sprawdza, czy klucz znajduje się w słowniku.

👉 not in – sprawdza, czy klucza nie ma w słowniku.



pl.pinterest.com/pin/412572015891911420/

```
ceny = {"jabłko": 3, "gruszka": 4}
print(ceny["jabłko"])
```

```
3
```

- for klucz in slownik: – przechodzi po kluczach.

- slownik.items() – umożliwia przechodzenie po parach klucz-wartość.

- slownik.values() – umożliwia przechodzenie po samych wartościach.

Zasada DRY

👉 DRY (Don't Repeat Yourself)
nie powtarzaj się. Warto unikać powielania tego samego kodu, np. przez używanie funkcji i wspólnych fragmentów programu.



pl.pinterest.com/pin/1023302346608031927/